[University name]
[Faculty / Institute – IACS]

Bachelor thesis

Latency and Accuracy Optimization for OCR to LLM Document Processing Pipelines

Design and comparison of a sequential and an optimised asynchronous OCR → LLM architecture for invoice data extraction

Author	Fleming Fredy Silva
Study programme	[study programme]
Supervisor	[supervisor's name and title]
Thesis type	Type 2 – solution to a current problem

[City], 2026

Abstract

OCR → LLM pipelines, which recognise the text of a document image and let a language model read the fields from it, are a common way to automate invoice processing. Run sequentially, one document and one stage at a time, they leave most of the hardware idle and become slow and expensive at volume. This thesis designs, implements and compares two architectures that extract the invoice number, date and total from invoice images on a laptop with a 4 GB GPU (NVIDIA RTX 3050). Architecture A is a sequential baseline: Tesseract OCR, a MiniLM embedding filter that keeps the relevant lines, and Flan-T5-base in fp32. Architecture B uses the same models, but runs OCR in parallel processes, overlaps OCR with GPU work through an asyncio event loop, batches the model calls, caches results by content hash, and routes each document to a 4-bit NF4 or an fp32 model with thresholds derived from data.

On 600 invoices (300 real, 300 synthetic, three runs per configuration), B processes 301.7 invoices per minute against 53.1 for A (5.7×). It lowers the cost per 100 invoices from €0.0110 to €0.0019 at an identical F1 of 0.941, using 1.9× the GPU memory. Parallel OCR (2.8×) and batching (2.1×) produce the gain. 4-bit quantisation saves a third of the GPU memory but no time at this model size, and the cache helps only repeated documents. With invoices arriving one at a time, fixed batches worsen latency at light load (7.9 s against 1.7 s median), and dynamic batching removes this penalty (0.95–1.8 s). Rebuilding the OCR text row by row and checking the total arithmetically raised F1 from 0.85 to 0.94. A vision-language model (Qwen2-VL-2B, 4-bit) on the same GPU is more accurate (F1 0.966) but 19 times slower per invoice.

Keywords: OCR, large language models, invoice extraction, asynchronous pipelines, batching, caching, quantisation, vision-language models.

Contents

Introduction 4

1 Analysis and review 4

1.1 Performance issues of OCR → LLM pipelines 4

1.2 OCR tools 5

1.3 Is OCR still needed? Vision-language models 6

1.4 Caching, compression and asynchronous processing 6

1.5 Technology choices and requirements 6

2 Design of the two architectures 7

2.1 Shared components 7

2.2 Architecture A – sequential baseline 8

2.3 Architecture B – optimised asynchronous pipeline 8

3 Dataset and evaluation method 10

3.1 Dataset and ground truth 10

3.2 Metrics 10

3.3 Experiment design 11

4 Results 11

4.1 Architecture A compared with Architecture B 11

4.2 Where the speed comes from 12

4.3 Memory and cost 13

4.4 Accuracy and human review 13

4.5 Quantisation, routing and cache 14

4.6 Latency with invoices arriving one at a time 15

4.7 OCR engines 16

4.8 Vision-language model on the same GPU 16

5 Discussion 17

5.1 Answers to the research questions 17

5.2 Trade-offs and relation to other work 17

5.3 Proposed improvements 18

Conclusions 18

References 19

Introduction

Every invoice a company receives has to be recorded with its number, date and amount, and much of this is still typed by hand. A popular way to automate it combines optical character recognition (OCR), which turns the image into text, with a language model that reads the text and returns the requested fields. Such OCR → LLM pipelines are easy to build from open-source parts, and their intermediate text can be inspected and checked. Run naively, however, they are slow. OCR is CPU-heavy and takes around a second per scanned page, the model runs on the GPU, and when the stages run one after another for one document at a time, each processor waits for the other. The model is often called once per field and loaded in full precision. Known remedies, such as parallelism, batching, caching and quantisation, are well described for large serving systems (Crankshaw et al., 2017; Kwon et al., 2023), but there is little evidence on what they deliver together for a complete document pipeline on modest hardware, and whether they change accuracy. Vision-language models (VLMs) that read images directly raise the further question whether OCR is needed at all.

Goal. To design and compare two OCR → LLM document-processing architectures, a simple sequential pipeline and an optimised asynchronous pipeline with caching and model compression, in order to improve latency, throughput and cost-efficiency while keeping high accuracy.

Tasks. (1) Analyse existing OCR → LLM pipelines and identify key performance issues such as latency, cost and scalability. (2) Review OCR tools, caching methods, compression techniques and asynchronous processing approaches. (3) Develop two prototypes: Architecture A, a baseline sequential OCR → Embeddings → LLM pipeline, and Architecture B, an optimised asynchronous pipeline with caching, batching and quantised models. (4) Describe both architectures: modules, data flow, and the integration of caching and compression. (5) Evaluate both on about 600 real and synthetic documents with accuracy (F1), latency, throughput, memory and cost. (6) Compare the results, discuss trade-offs and propose improvements.

Research questions. RQ1: How much can the optimised architecture improve throughput, latency, memory and cost over the sequential baseline with the same models and hardware, and does accuracy stay the same? RQ2: Which optimisations account for the improvement? RQ3: How does the OCR → LLM approach compare with a VLM that reads the image directly on the same GPU?

Scope. The task is to extract three header fields (invoice number, date and total amount due) from single-page invoice images. All measurements were taken on one laptop with an NVIDIA GeForce RTX 3050 Laptop GPU (4 GB), 12 logical CPU cores and 16 GB of RAM. Both architectures share every component that produces an answer, so differences in speed, memory and cost can be attributed to the architecture. Chapter 1 analyses the problem and reviews the technologies (tasks 1–2). Chapter 2 describes the design (tasks 3–4), and Chapter 3 the dataset and evaluation method. Chapter 4 reports the results (task 5), and Chapter 5 discusses them (task 6).

1 Analysis and review

1.1 Performance issues of OCR → LLM pipelines

A typical OCR → LLM extraction pipeline has five stages: image pre-processing, OCR (words with bounding boxes and confidences), context selection (reducing the text to what the model can read), extraction by the language model, and validation of the answers. Key information extraction of this kind has moved from templates and rules, through machine learning on OCR text (Palm et al., 2017) and layout-aware transformers (Xu et al., 2020; Huang et al., 2022), to generative models that answer questions about the text (Luo et al., 2024; Lamott et al., 2024). Analysing such pipelines (task 1) reveals four groups of issues:

- **Latency.** In a sequential pipeline latency is the sum of the stage times. OCR alone takes about 2 s per page with Tesseract and 10–15 s through cloud OCR APIs (Hegghammer, 2022), and each model call adds a fixed overhead. In batched or queued systems latency also includes waiting: by Little's law the

queue grows without bound once arrivals exceed the service rate (Little, 2011), and batching raises throughput "at the expense of increased latency" (Crankshaw et al., 2017).

- **Throughput and utilisation.** A sequential pipeline uses one resource at a time: the GPU is idle during OCR and the CPU cores are idle during inference. Small, separate model calls also use the GPU poorly, because their run time is dominated by loading weights rather than computing (Williams et al., 2009). When stages overlap, the slowest stage limits the speed-up (Amdahl, 1967).
- **Cost and scalability.** Cost is proportional to the hardware time per document, so idle hardware is paid for. API-based VLM extraction is far more expensive than self-hosted pipelines (Poznanski et al., 2025). On one machine, scaling is limited by CPU cores for OCR, by GPU memory, about 4 GB per billion parameters in fp32 (Hugging Face, n.d.), and by Python's global interpreter lock, which prevents CPU-bound threads from running in parallel (Python Software Foundation, n.d.-b).
- **Accuracy risks.** OCR errors propagate to extraction (van Strien et al., 2020; Anvari, 2026), losing the layout separates labels from values (Lamott et al., 2024), and generative models can return values that are not in the document (He et al., 2025). An optimisation is only acceptable if accuracy is measured with it.

The first prototype of this project showed several of these issues and some measurement errors: it asked Flan-T5 for JSON, which the model cannot produce because its vocabulary has no braces; it computed throughput from summed per-document times, which hides any parallelism; and it ran A on the CPU and B on the GPU. The final design avoids all three (Table 1.1).

Table 1.1. Performance issues and the remedies examined in this thesis.

Issue	Remedy examined	Where
Idle GPU during OCR, idle CPU during inference	overlap OCR processes with a GPU thread	Section 2.3
OCR limits throughput	several OCR worker processes	Section 4.2
Many small model calls	batch fields and documents into one call	Section 4.2
Repeated documents recomputed	content-addressed result cache	Section 2.3.1
GPU memory	fp16, 8-bit and 4-bit weights	Section 4.5
Same cost for easy and hard documents	routing by token count and OCR confidence	Section 2.3.2
Queueing latency	dynamic batching	Section 4.6
Layout and net/gross errors	row reconstruction, total check, review flag	Section 2.1

1.2 OCR tools

Tesseract (Smith, 2007) runs on the CPU, uses an LSTM line recogniser since version 4, and returns every word with its box and a confidence in one call. It is sensitive to noise but competitive on clean print (Hegghammer, 2022; Hammarström, 2025), and it was the fastest engine in a recent comparison at 0.58 s per image (Nagayi et al., 2025). **EasyOCR** (JaidedAI, n.d.) combines a neural detector and recogniser on the GPU, but its character error rate on receipts was the highest of the open-source engines tested by Hemmer et al. (2024), and in the Docling toolkit it is "the biggest compute expense in the pipeline" (Livathinos et al., 2025). **PaddleOCR** (Li et al., 2022) and **docTR** (Mindee, 2021) are lighter neural alternatives, and **cloud OCR** is the most accurate on noisy scans but is billed per page and sends documents off the machine. OCR confidence is a useful signal of quality (Hemmer et al., 2024), which supports using it for routing and review.

1.3 Is OCR still needed? Vision-language models

OCR-free models such as Donut (Kim et al., 2022) and Pix2Struct (Lee et al., 2023), and general VLMs such as Qwen2-VL (Wang et al., 2024), read the document image directly. Their cost is resolution: small text needs large images, which become thousands of visual tokens (Shpigel Nacson et al., 2025), and document accuracy rises steeply with the token count (Beyer et al., 2024). At default settings, Qwen2-VL-2B was measured at 13.7 GB of GPU memory (Marafioti et al., 2025). Evidence that OCR text still helps is strong: adding it improved GPT-4 Vision on DocVQA (Borchmann, 2024), text-only LLMs on OCR text beat VLMs of the same size in zero-shot extraction (Luo et al., 2024), and an OCR-based multi-stage pipeline reached 8.8 times the field accuracy of a 72-billion-parameter VLM at 0.7 % of its GPU cost (Jin et al., 2025). VLMs also hallucinate values (He et al., 2025). On the other hand, image-only Gemini 1.5 Pro was the most accurate system on about 1,000 real business documents, with the conclusion that "OCR may not be necessary for powerful MLLMs" (Shen et al., 2026). The defensible position is conditional. Under a 4 GB GPU, local processing and auditable intermediate text, an OCR → LLM pipeline is justified, but mainly for speed, memory and cost. This thesis therefore measures a VLM on the same GPU (Section 4.8).

1.4 Caching, compression and asynchronous processing

Caching. Serving systems cache predictions keyed on the model and the input (Crankshaw et al., 2017). Semantic caches answer *similar* queries (Bang, 2023) and risk false hits, so for documents an exact cache keyed on a SHA-256 hash of the content (NIST, 2015) is safer. SQLite keeps such a cache in one durable file without a server (SQLite, n.d.); Redis pays off only when several machines share it.

Compression. Half precision halves weight memory. Post-training quantisation stores weights in 8 bits with LLM.int8(), which keeps outliers in 16 bits (Dettmers et al., 2022), or in 4 bits with QLoRA's NF4 data type (Dettmers et al., 2023), both in bitsandbytes; GPTQ and AWQ need calibration (Frantar et al., 2023; Lin et al., 2024). Weight-only quantisation de-quantises every matrix before multiplying, so inference time "is often not reduced … but rather increases" (Hugging Face, n.d.), especially for small models (Belkada and Dettmers, 2022); it pays off mainly when the full model does not fit the GPU (Arte, 2024).

Asynchronous processing and batching. `asyncio` interleaves waiting tasks but does not speed up computation, so CPU-bound work must run in a process pool via `run_in_executor` (Python Software Foundation, n.d.-a). Batching raises throughput at the cost of waiting; dynamic batching sends a batch when it is full or a delay expires (NVIDIA, n.d.), continuous batching serves large decoders (Yu et al., 2022; Kwon et al., 2023), and cascades send easy requests to cheap models (Chen et al., 2023). The closest comparable thesis reached a 5.33× throughput gain for transformer batch inference through batching and CPU/GPU overlap (Juusti, 2026). No study was found that compares a sequential and an optimised OCR → LLM pipeline on consumer hardware with accuracy and a VLM baseline, which is the gap this thesis addresses.

1.5 Technology choices and requirements

Table 1.2 justifies each technology choice against its alternatives. The requirements derived from the analysis are: (R1) extract the three fields as JSON from PNG, JPG and WebP images; (R2) A and B share OCR, filter, model, prompts, post-processing and scorer and run on the same GPU; (R3) everything runs offline within 4 GB of GPU memory; (R4) routing thresholds are derived from measurements; (R5) accuracy is measured as F1 with fuzzy matching (≥ 90), and performance as latency, throughput, memory and cost per 100 documents, over three runs per configuration; (R6) uncertain documents are flagged for human review; (R7) every run is reproducible.

Table 1.2. Technology decisions.

Decision	Chosen	Alternatives	Reason
OCR	Tesseract 5.4	EasyOCR, PaddleOCR, cloud	CPU only, so the GPU is free for the models; boxes and confidence in one call; measured faster and more accurate here (Section 4.7)
Model	Flan-T5-base (248 M)	larger LLMs, a VLM	fits 4 GB in every precision incl. fp32, so A vs B compares architecture, not memory starvation
Context	MiniLM line filter	truncation	the total is near the bottom; truncation would drop it
Prompt	one question per field	one JSON prompt	T5 cannot emit braces; QA is its training format
Cache	SQLite, SHA-256 key	Redis, Memcached	one machine, no server, durable, ≈ 1 ms per lookup
Concurrency	asyncio + process pool + 1 GPU thread	Celery, Kafka	single node; processes bypass the GIL; no broker needed
Quantisation	bitsandbytes NF4 (and 8-bit, fp16 as variants)	GPTQ, AWQ, GGUF	integrated in transformers, no calibration data

2 Design of the two architectures

Both architectures answer one question: how much can the *execution architecture* save for a fixed extraction method? They therefore use identical extraction components on the same GPU, are warmed up before timing, and are measured by wall-clock time. Only the scheduling and the precision of the model on B's fast route differ.

2.1 Shared components

OCR with row reconstruction. Tesseract 5 is called once per image through `image_to_data`, which returns every word with its box, line number and confidence (0–100). Tesseract orders text by blocks, so in tables a label and its amount can end up 20–40 lines apart. The text is therefore rebuilt row by row (Algorithm 1). The mean word confidence is kept for routing and review.

```
Algorithm 1 – Row reconstruction
segments ← Tesseract words grouped by (block, paragraph, line), box = union of word boxes
tol ← 0.6 · median segment height;  sort segments by vertical centre
for each segment s: if |centre(s) – centre(current row)| ≤ tol: add s to the row  else: start a new
 row
emit each row left to right, joined by " | "   (e.g. Total | $ 297,35 | $ 29,73 | $ 327,08)
```

Line filter. Flan-T5 reads at most 512 tokens, and the total is usually at the bottom of the page, so the text cannot simply be truncated. all-MiniLM-L6-v2 embeds every line, and for each of the queries "invoice number", "invoice date" and "total amount due" the filter keeps the 6 most similar lines plus the line following each, because values often sit below their labels. Texts of at most 18 lines pass unchanged.

Extraction. `google/flan-t5-base` (248 M parameters) answers one reading-comprehension question per field (`<lines> Question: What is the total amount due? Answer:`) with greedy decoding. A JSON prompt cannot work, because T5's vocabulary has no `{` or `}`. The model can be loaded in fp32 (≈ 990 MB of weights), fp16 (≈ 500 MB), 8-bit LLM.int8 (≈ 300 MB) or 4-bit NF4 (≈ 200 MB) through bitsandbytes:

```
BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                   bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=torch.float16)
```

Normalisation, total check and review flag. Invoice numbers, dates and amounts are reduced to a canonical form, for example `INV-8289-16` → `INV828916`, `188,11` → 188.11 and `FEB112020` → 11 Feb 2020, which is used for output and scoring alike. Because the model often returns the net amount (the first amount after "Total"), a reconciliation step collects the amounts on lines containing *total*, *amount due*, *balance due* or *gross*: if two of them add up to a third (subtotal + tax = total, ± 0.01), the third is returned. Finally, an invoice is flagged for human review when a field is missing or unparsable, the total does not appear on a total line, or the mean OCR confidence is below 80.

2.2 Architecture A – sequential baseline

Architecture A processes one invoice at a time in one process (Figure 2.1): OCR on the CPU (≈ 0.59 s), the line filter on the GPU, three `generate` calls in fp32 (≈ 0.51 s), then validation. Nothing overlaps, so its latency equals its service time of about 1.1 s per invoice.

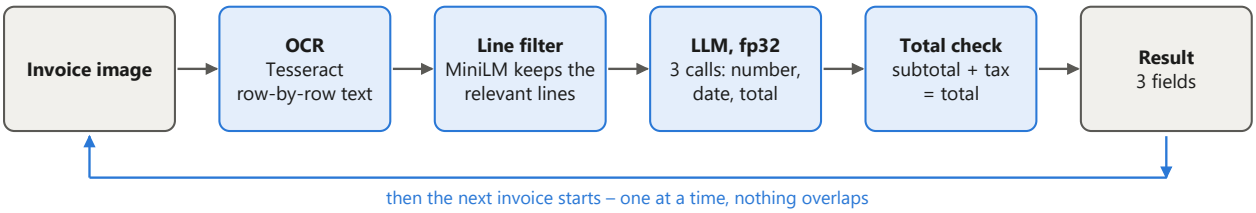


Figure 2.1. Architecture A: one invoice passes through all stages before the next one starts.

2.3 Architecture B – optimised asynchronous pipeline

Architecture B (Figure 2.2) first looks up every document in the cache. The remaining documents flow through a pipeline in which OCR and the GPU work on different invoices at the same time:

- An **asyncio event loop** submits OCR jobs to a **process pool of 3 workers** with `run_in_executor`, collects results in completion order, and hands every 8 results as a **batch** to a **single GPU thread**. While the GPU works on batch *k*, the workers already read batch *k* + 1. Processes bypass the GIL, and one GPU thread avoids two concurrent model calls competing for 4 GB.
- The GPU thread runs the line filter for the whole batch in one call, **routes** each invoice, and runs one `generate` call per route with up to 24 prompts (8 invoices × 3 questions). It then applies the total check and the review flag, and writes the results to the cache.

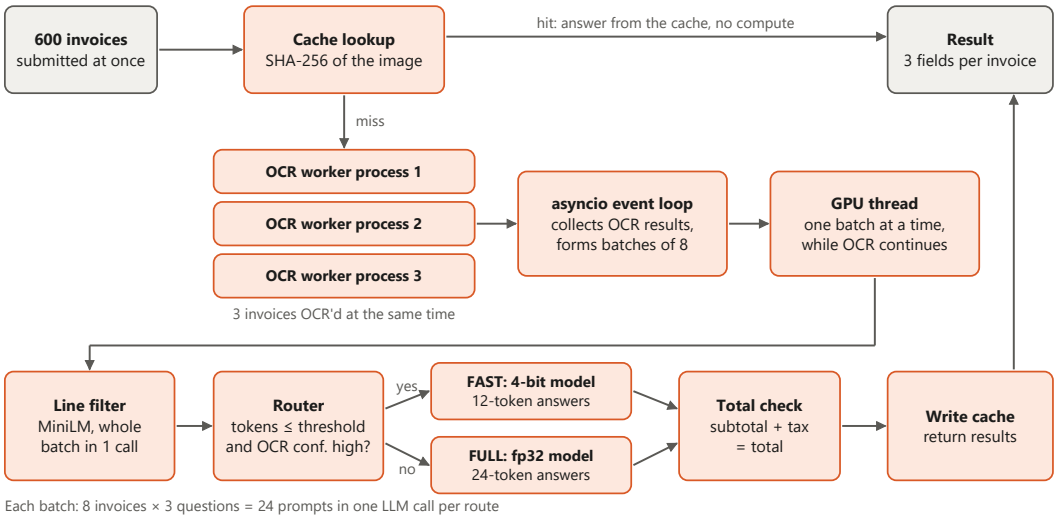


Figure 2.2. Architecture B: cache lookup, three OCR worker processes, an event loop that forms batches, and one GPU thread that filters, routes and extracts a batch at a time.

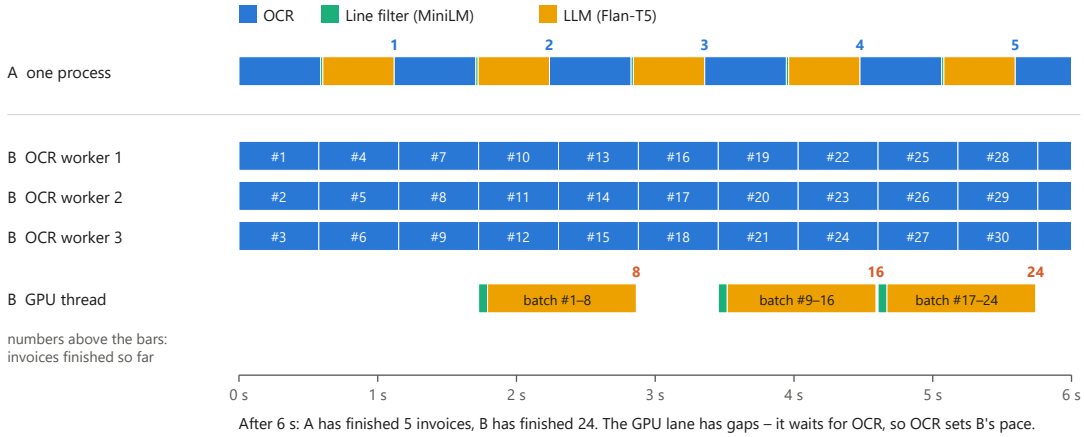


Figure 2.3. First 6 s of both architectures, drawn to scale from the mean stage times of the benchmark. In B, OCR runs continuously in three processes while the GPU processes batches of 8; the gaps in the GPU lane show that OCR sets B's pace.

2.3.1 Integration of caching

The cache is a SQLite table keyed by a SHA-256 digest of the configuration and the image bytes:

$$key(d) = SHA\text{-}256(\ model \parallel precisions \parallel routing\ policy \parallel thresholds \parallel extraction\ version \parallel bytes(d)\ ) \tag{2.1}$$

A renamed copy of an invoice therefore hits the cache, while a changed image or configuration misses, so results cannot leak between experiments. The stored payload is JSON (prediction, route, features), and each output folder has its own cache file.

2.3.2 Integration of compression and routing

B keeps two copies of Flan-T5 on the GPU: a 4-bit NF4 copy for the FAST route (at most 12 answer tokens) and an fp32 copy for the FULL route (24 tokens). With $t(d)$ the OCR text length in Flan-T5 tokens and $c(d)$ the mean OCR confidence of document d:

$$route(d) = FAST \text{ if } t(d) \le T \text{ and } c(d) \ge C;\ FULL \text{ otherwise} \tag{2.2}$$

The thresholds are derived from data rather than guessed. The dataset is run once entirely on FAST and once entirely on FULL, which gives each document's field accuracy under both routes. A grid search over T from 50 to 800 tokens and C from 70 to 95 then selects

$$(T_{opt}, C_{opt}) = \operatorname{argmax} share_{FAST}(T, C) \text{ subject to } acc(T, C) \ge acc_{FULL} - 0.005 \tag{2.3}$$

that is, the largest share of documents on the cheaper route whose accuracy stays within 0.5 percentage points of all-FULL. Configuration B uses the initial estimate T = 300, C = 85; configuration B-tuned uses the derived values.

2.3.3 Latency and dynamic batching

B records each invoice's *service time* (OCR plus its share of the batch's filter and model time), comparable with A, and its *latency* from submission to result, which includes waiting. When invoices arrive over time, B as designed waits until 8 are read. With **dynamic batching** (`max_batch_wait_s = 0`) the GPU instead takes every invoice already read whenever it is free, so batches are small at light load and grow under heavy load, like the queue delay of Triton's dynamic batcher (NVIDIA, n.d.).

The implementation (Python 3.12, PyTorch 2.5.1, transformers 5.7, bitsandbytes 0.50; about 3,300 lines, 66 unit tests) writes every run to a new folder with settings, hardware and per-invoice results.

3 Dataset and evaluation method

3.1 Dataset and ground truth

The dataset has 600 invoice images, each with a JSON ground-truth file. 300 are **real** invoices (numbers 1–15 and 316–600): public invoice templates and scans of 1131 × 1600 to 2481 × 3509 pixels in WebP, PNG and JPG, labelled by hand, many with line-item tables and a net/tax/gross summary row. The other 300 are **synthetic** (numbers 16–315): clean 800 × 600 or 800 × 1000 images rendered by a generator with a company name, the three fields, optional line items and distracting lines (Figure 3.1). The synthetic half is a sanity check and the real half the actual test, so accuracy is always reported for both halves. The `total` label means the gross amount due. During the error analysis, 26 real-invoice labels turned out to hold the net amount; each was changed to the gross only when net + tax = gross held exactly on the OCR text, and the originals were kept.

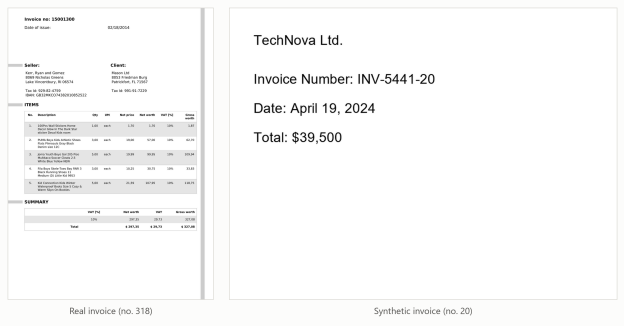


Figure 3.1. A real invoice (left), whose summary row holds net, tax and gross side by side, and a synthetic invoice (right).

3.2 Metrics

Accuracy. Each field is compared with its ground truth after normalisation. A correct value is a true positive (TP); a wrong value counts as both a false positive (FP) and a false negative (FN); a missing value is an FN. Invoice numbers match when their canonical forms are equal or their RapidFuzz similarity is at least 90, which tolerates one OCR character error. Dates match when they denote the same calendar day, and amounts when they differ by less than 0.005. Precision, recall and F1 are micro-averaged over all fields (Manning et al., 2008):

$$P = TP / (TP + FP),\ R = TP / (TP + FN),\ F1 = 2 \cdot P \cdot R / (P + R) \tag{3.1}$$

The share of invoices with all three fields correct (*fully correct*) is reported as well.

Performance. *Wall time* runs from submitting the first invoice to receiving the last result, with models already loaded; *throughput* is invoices per minute of wall time; *service time* is the compute per invoice; *latency* is the time from submission (or arrival) to result, reported as median and 95th percentile. *Peak RAM* covers the whole process tree and *pipeline GPU memory* is PyTorch's peak allocation, both sampled four times per second. *Cost* is modelled as

$$\text{cost per 100 invoices} = 100 \cdot (\text{wall time} / N) \cdot €0.35 \text{ per GPU-hour} / 3600\ s \tag{3.2}$$

using a typical price for an entry-level cloud GPU. The ratio between A and B does not depend on that price.

3.3 Experiment design

Each configuration processes all 600 invoices three times; the tables give mean ± standard deviation. Besides A and B, nine variants of B change exactly one factor each (Table 3.1).

Table 3.1. Benchmark configurations.

Configuration	What changes in B
B-warm	the same invoices again with a full cache
B-nocache	no cache
B-allfull / B-allfast	every invoice on the fp32 / on the 4-bit model
B-fp16 / B-8bit	every invoice on an fp16 / an LLM.int8 model
B-1worker / B-6workers	1 / 6 OCR worker processes instead of 3
B-nobatch	batch size 1 instead of 8
B-tuned	routing thresholds derived from B-allfast and B-allfull

All experiments ran on the laptop described in the introduction (Intel 12th-generation CPU, Windows 11, Python 3.12.2, PyTorch 2.5.1 with CUDA 12.1, transformers 5.7.0, bitsandbytes 0.50.2, Tesseract 5.4.0). The laptop was plugged in and kept awake, models were loaded offline, and nothing else ran during the measurements. Every run was checked by its mean OCR time per invoice (normally 0.55–0.63 s). Runs disturbed by other programs, including two overlapping demo runs and a runaway background process, were moved to a `discarded/` folder and repeated: 33 runs in total.

Additional experiments. (1) *Latency*: 60 invoices arrive at random (Poisson) times at 30, 48, 120 and 240 per minute, three runs each, for A, for B as designed, and for B with dynamic batching. (2) *OCR engines*: Tesseract (CPU) and EasyOCR (GPU) read all 600 invoices, and each engine's text goes through the identical downstream of A. (3) *VLM baseline*: Qwen2-VL-2B-Instruct reads the image only and returns the fields as JSON, loaded in 4-bit because its 16-bit weights (4.4 GB) do not fit the GPU, with images capped at 1,024 visual tokens. Its answers go through the same normalisation and scorer as A and B.

4 Results

All numbers are the mean ± standard deviation of three runs over 600 invoices from the benchmark folder `outputs/2026-10-18_23-52-41_full_benchmark`, unless another experiment is named.

4.1 Architecture A compared with Architecture B

Table 4.1. A and B on 600 invoices.

Measure	A (sequential)	B (optimised)	B vs A
Wall time for 600 invoices	677.9 ± 7.4 s	119.4 ± 4.7 s	5.7× faster
Throughput	53.1 ± 0.6 /min	301.7 ± 11.8 /min	5.7×
Service time per invoice (OCR / filter / LLM)	1.119 s (0.588 / 0.017 / 0.514)	0.718 s (0.576 / 0.008 / 0.134)	1.6× lower
Pipeline GPU memory / peak RAM	1067 / 1438 MB	1996 / 2012 MB	1.9× / 1.4× higher
Cost per 100 invoices	€0.0110	€0.0019	5.7× lower
F1 all / real / synthetic	0.941 / 0.892 / 0.989	0.941 / 0.894 / 0.987	equal

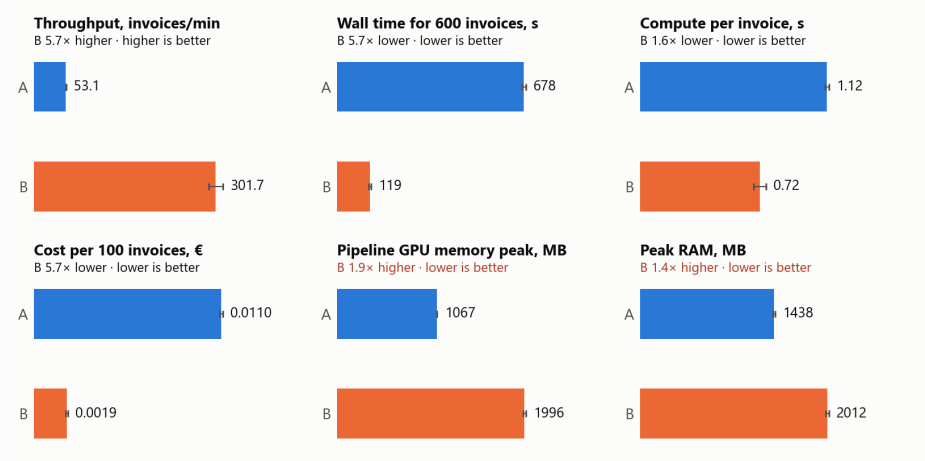


Figure 4.1. A and B on six performance measures (mean of 3 runs; whiskers: standard deviation). Red subtitles mark where B is worse.

B finishes the 600 invoices 5.7 times faster at identical accuracy (Table 4.1, Figure 4.1). Throughput improves far more than compute per invoice (1.6×) because B overlaps stages: three OCR processes and the GPU work on different invoices at the same time. OCR costs the same in both architectures, while the LLM step is 3.8 times cheaper per invoice in B, because 24 prompts share one call. B pays with memory, two model copies and three worker processes, but stays well within 4 GB of GPU memory (whole-card peak 3.2 GB) and 16 GB of RAM. Its first result arrives after 4.4 s instead of 0.6 s, because it waits for a full batch; for a single invoice both need about 1.3 s.

4.2 Where the speed comes from

Table 4.2. Ablation: each variant changes one part of B.

Configuration	Wall time for 600	Effect
B (3 OCR workers, batch 8)	119.4 ± 4.7 s	reference
B-1worker	338.4 ± 5.1 s	parallel OCR is worth 2.8×
B-nobatch	248.8 ± 2.5 s	GPU batching is worth 2.1×
B-6workers	102.9 ± 10.3 s	16 % faster, unstable, 2.6 GB RAM
B-nocache	126.0 ± 2.6 s	cache lookup costs nothing measurable

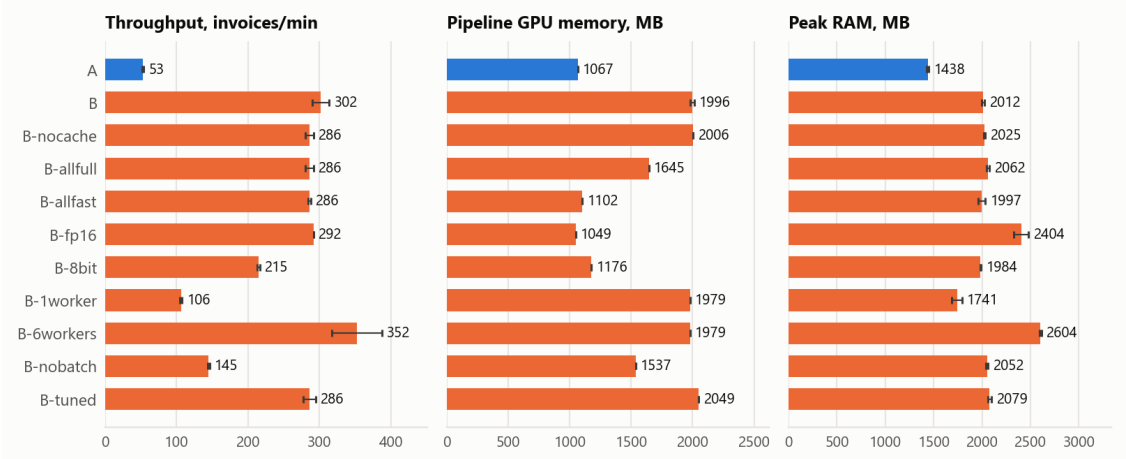


Figure 4.2. Throughput, pipeline GPU memory and peak RAM of every configuration (mean of 3 runs; whiskers: standard deviation). B-warm (0.59 s for all 600 invoices) is omitted.

Parallel OCR and GPU batching produce the speed-up (Table 4.2, Figure 4.2). A simple bottleneck model explains every variant. With *N* invoices, *w* OCR workers, mean OCR time *o* and mean filter and model time *e* + *g* per invoice, the two overlapping stages need *N·o/w* and *N·(e + g)* seconds, and the slower one sets the pace:

$$W_B \approx max(N \cdot o / w , N \cdot (e + g)) \quad (4.1)$$

For B this predicts max(115 s, 85 s) = 115 s against 119.4 s measured, so B is **OCR-bound**. Without batching or with 8-bit weights the GPU becomes the slower stage. The model matches all variants within about 8 % (Figure 4.3), and it explains why a faster LLM step barely changes B's wall time.

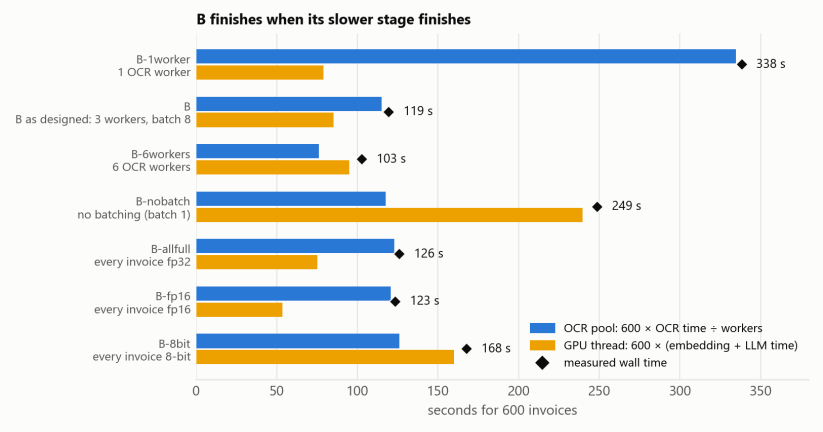


Figure 4.3. Time each stage of B needs for 600 invoices (from mean stage times) and the measured wall time.

4.3 Memory and cost

B's GPU memory is higher because the fp32 and 4-bit models are resident together; with one model only, it falls to 1102 MB (4-bit) or 1049 MB (fp16). RAM grows with the number of OCR workers: 2012 MB with three, 2604 MB with six (Figure 4.2). Cost follows wall time: €0.0110 per 100 invoices for A, €0.0019 for B and €0.0055 for B-1worker. For a million invoices that is about 314 against 55 hours of machine time.

4.4 Accuracy and human review

Table 4.3. Accuracy before and after the extraction upgrade (identical in all runs).

Extraction	Arch.	F1 all	F1 real	F1 synth.	Real: number / date / total	Real fully correct
Block-order text, model's total	A	0.847	0.704	0.989	0.93 / 0.86 / 0.32	24 %
	B	0.850	0.712	0.987	0.93 / 0.86 / 0.34	26 %
Row text + total check (final)	A	0.941	0.892	0.989	0.94 / 0.92 / 0.82	72 %
	B	0.941	0.894	0.987	0.94 / 0.92 / 0.83	72 %

A and B are equally accurate by design: 505 of 600 invoices (84.2 %) are fully correct in both. The first benchmark, however, reached only F1 0.85, with a third of the real totals correct. In 154 of the 199 wrong totals the right amount was already in the model's input. Tesseract's column-wise text had separated "Total" from its amount, and in rows such as `Total | $ 297,35 | $ 29,73 | $ 327,08` the model took the net amount. Row reconstruction and the total check fixed this in both architectures and raised F1 to 0.941 (Table 4.3, Figure 4.4). Of the 146 totals that turned correct, 121 belong to invoices whose labels were never changed, so the gain does not depend on the label corrections.

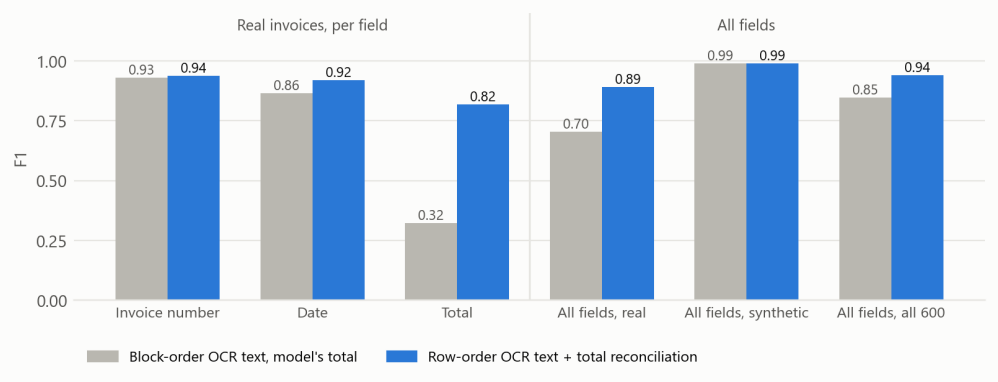


Figure 4.4. F1 of Architecture A before and after the extraction upgrade, per field on real invoices (left) and over all fields (right).

Human review. B accepts 92.5 % of the invoices automatically, and 90.5 % of those are fully correct, against 84.2 % if every invoice were accepted. Of the 45 flagged invoices, 42 (93 %) really were wrong. The flag is therefore precise, but it catches only 42 of the 95 wrong invoices: an invoice number or date copied from a neighbouring field passes every rule.

4.5 Quantisation, routing and cache

Table 4.4. B with every invoice on one model in four precisions.

Precision	Wall time for 600	LLM per invoice	GPU peak	F1
fp32 (B-allfull)	126.1 ± 2.6 s	0.115 s	1645 MB	0.941
fp16 (B-fp16)	123.5 ± 0.1 s	0.079 s	1049 MB	0.941
4-bit NF4 (B-allfast)	125.8 ± 0.8 s	0.115 s	1102 MB	0.941
8-bit LLM.int8 (B-8bit)	167.8 ± 1.6 s	0.258 s	1176 MB	0.9385 ± 0.0003

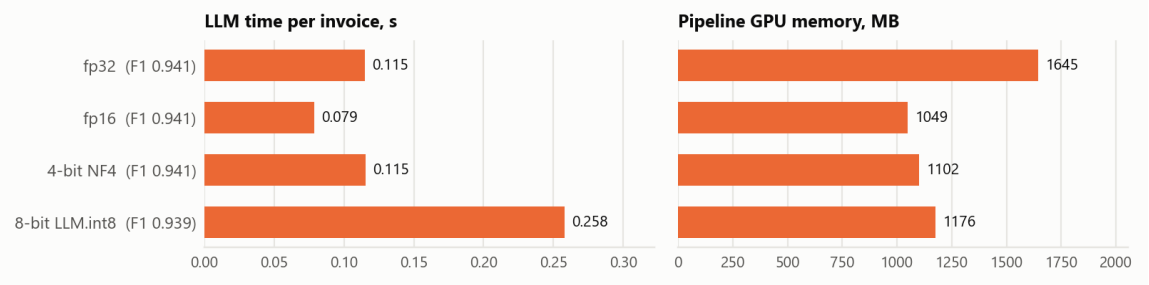


Figure 4.5. LLM time per invoice (left) and pipeline GPU memory (right) of B with every invoice on one model, per precision.

4-bit saves memory, not time: it uses a third less GPU memory than fp32 at the same speed and F1 (Table 4.4, Figure 4.5). **fp16 is best** on this GPU, with the fastest model step and the least memory. **8-bit is slowest** (2.2× fp32) and slightly non-deterministic. For a 248-million-parameter model, de-quantisation costs as much as the memory traffic it saves.

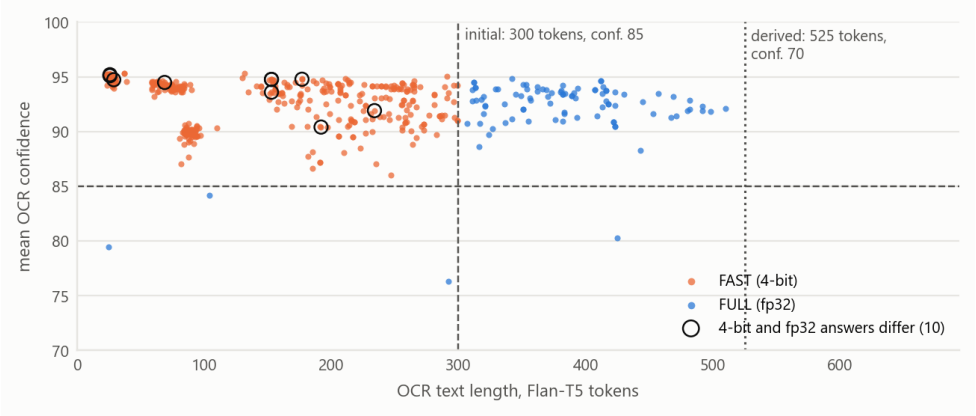


Figure 4.6. Routing per invoice with the initial thresholds (dashed: 300 tokens, confidence 85) and the derived token threshold (dotted: 525). Circles mark the 10 invoices where the 4-bit and fp32 answers differ; one invoice with OCR confidence below 70 is not shown.

With the initial thresholds, B routes 505 invoices to FAST and 95 to FULL (Figure 4.6). The routing analysis (Equation 2.3) found that the 4-bit and fp32 models disagree on only 10 invoices, 5 each way, with equal accuracy in every token and confidence bin. The derived thresholds are therefore T = 525 and C = 70, which send 99.8 % of invoices to FAST, and three fresh runs (B-tuned) confirm F1 0.941 at 126.1 s. The router works, but two precisions of one small model leave it little to choose between. **Cache:** the same 600 invoices submitted again take 0.59 ± 0.03 s, about 1 ms per invoice, and on a first pass the cache costs nothing measurable.

4.6 Latency with invoices arriving one at a time

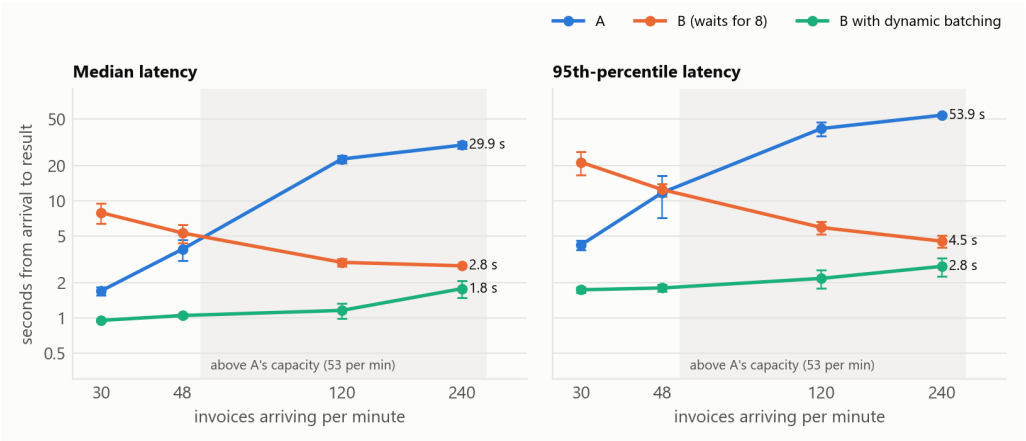


Figure 4.7. Latency from arrival to result for 60 invoices arriving at random times (mean of 3 runs; log scales). The grey area is above A's capacity.

Table 4.5. Median latency in seconds (mean ± standard deviation of 3 runs; `outputs/2026-10-20_01-01-33_latency_60docs`).

Arrivals per minute	A	B (waits for 8)	B, dynamic batching
30	1.70 ± 0.13	7.94 ± 1.57	0.95 ± 0.03
48	3.87 ± 0.79	5.31 ± 0.93	1.05 ± 0.04
120	22.76 ± 1.64	2.98 ± 0.22	1.16 ± 0.18
240	29.94 ± 2.03	2.79 ± 0.02	1.78 ± 0.29

A answers quickly while it keeps up, but above about 53 invoices per minute its queue grows without bound. B as designed waits for a batch of 8, so at light load it is slower than A (7.9 s against 1.7 s). Dynamic batching is the fastest at every load, with a median of 0.95–1.8 s and a 95th percentile below 2.8 s, and F1 is identical in all 36 runs (Figure 4.7, Table 4.5).

4.7 OCR engines

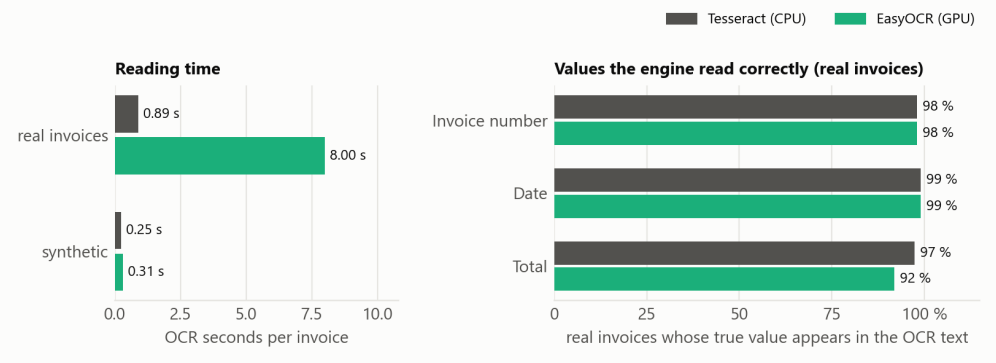


Figure 4.8. Left: OCR time per invoice. Right: share of real invoices whose true value appears in the OCR text (`outputs/2026-10-20_01-52-39_ocr_comparison`).

With the same downstream, Tesseract reaches F1 0.941 and EasyOCR 0.934 (real invoices 0.892 against 0.871). On the real scans EasyOCR needs 8.0 s per invoice on the GPU against 0.89 s for Tesseract on one CPU core, and its peak of 3.9 GB leaves no room for the language model. The true total appears in Tesseract's text for 97 % of real invoices and in EasyOCR's for 92 % (Figure 4.8). This experiment also exposed a date-parsing bug ("April 19,2024" read with the current year), which was fixed and all runs re-scored; the main benchmark was unaffected.

4.8 Vision-language model on the same GPU

Table 4.6. OCR pipelines and a VLM on the same 600 invoices and scorer (`outputs/2026-10-20_23-54-32_vlm_baseline`).

System	Time per invoice	GPU peak	F1 all	F1 real	Fully correct
A (OCR text)	1.13 s	1067 MB	0.941	0.892	505
B (OCR text)	0.20 s	1996 MB	0.941	0.894	505
Qwen2-VL-2B, 4-bit, ≤ 1,024 visual tokens (image)	3.90 s	1745 MB	0.966	0.930	541

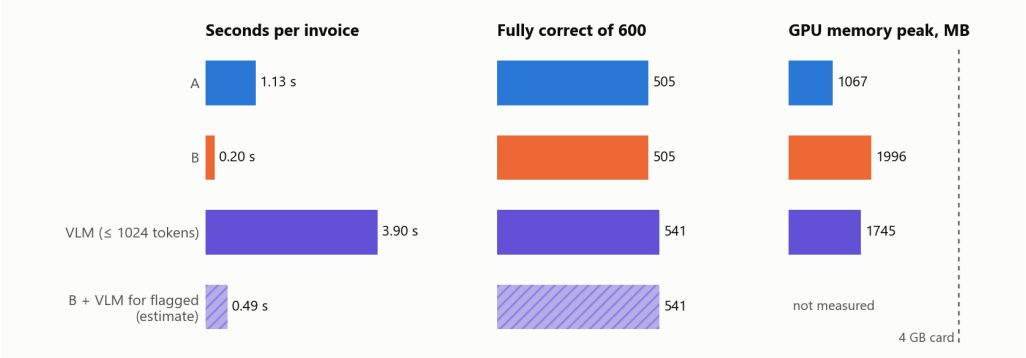


Figure 4.9. Time per invoice, fully correct invoices and GPU memory. The hatched row is an estimate: B's answers, with the VLM's answers for the invoices B flags for review.

The VLM is more accurate (F1 0.966, 541 fully correct invoices) but needs 3.9 s per invoice, 19 times B's time, and it fits the 4 GB GPU only in 4-bit (Table 4.6). A 20-invoice probe gave 1568, 1745 and 2329 MB at 512, 1,024 and 2,048 visual tokens, with no accuracy gain beyond 1,024. On the 45 invoices that B flags, B is fully correct 3 times and the VLM 39 times. Sending only those to the VLM is estimated to give 541 fully correct invoices, as many as the VLM alone, for about 0.29 s of extra GPU time per invoice (Figure 4.9).

5 Discussion

5.1 Answers to the research questions

RQ1. With the same models on the same laptop, Architecture B reaches 5.7 times the throughput and 5.7 times lower cost of Architecture A, at identical F1 (0.941), using 1.9 times the GPU memory and 1.4 times the RAM. Latency depends on the batching policy. With a fixed batch of eight, B is slower than A at light load; with dynamic batching it is the fastest at every load tested. The goal, better latency, throughput and cost at unchanged accuracy, is therefore met by B with dynamic batching.

RQ2. Parallel OCR (2.8×) and GPU batching (2.1×) produce the gain. Their overlap turns a 1.6-fold reduction of compute into a 5.7-fold gain in throughput, and the bottleneck model (Equation 4.1) predicts every variant within about 8 %. Quantisation does not pay off in time at this model size: 4-bit saves memory, fp16 is both faster and smaller, and 8-bit is slowest. The cache only helps repeated documents. Routing between two precisions of the same model changes little, because they almost always agree.

RQ3. A 2-billion-parameter VLM on the same GPU is more accurate but 19 times slower than B, and EasyOCR was slower and less accurate than Tesseract. On this hardware the OCR → LLM pipeline is justified by throughput, cost and its inspectable text, not by accuracy, and a cascade that sends only flagged invoices to the VLM combines both strengths.

5.2 Trade-offs and relation to other work

Throughput against latency. Batching serves many invoices per GPU call but makes each wait for its batch (Crankshaw et al., 2017); taking whatever is ready when the GPU is free resolves this, as in Triton's dynamic batcher (NVIDIA, n.d.). Fixed batches suit overnight bulk jobs; interactive uploads need dynamic batching. **Memory against speed.** Quantisation does not accelerate small models, in line with the library documentation (Hugging Face, n.d.) and LLM.int8() measurements (Belkada and Dettmers, 2022); it is the right tool when a model would not fit otherwise, as with the VLM (Arte, 2024). **Accuracy against cost.** The VLM gives 36 more fully correct invoices at 19 times the time, and a cascade keeps most of both (Ma et al., 2023; Chen et al., 2023). **Performance against complexity.** B has more moving parts, and one incident (a shared cache cleared by another run) showed the risk; content-addressed keys, per-run caches and unit tests contain it. The speed-up agrees with the 5.33× of Juusti (2026), the VLM result supports Shen et al. (2026), and the gain from row-wise text matches the layout effect reported by Lamott et al. (2024).

Threats to validity. Laptop timings are noisy; this was controlled by power settings, an OCR-time check of every run and 33 repeated runs, and differences of a few percent are not interpreted. The results cover one small model family, three header fields, mostly English single-page invoices and one machine; with more cores or a larger model the bottleneck, and the value of quantisation, would shift. Half of the data is synthetic, so results are reported per subset. 121 of the 146 totals fixed by the upgrade are on labels that were never changed. Cost is modelled from wall time only, and the OCR, VLM and cascade figures are single passes or estimates.

5.3 Proposed improvements

Table 5.1. Improvement options, ordered by the strength of the evidence.

Option	Effect	Basis
Dynamic batching as B's default	median latency 7.9 → 0.95 s at 30 invoices/min, same F1	measured
Faster OCR (more cores, faster CPU engine, downscaling large scans)	B's wall time follows OCR time (Equation 4.1)	measured
fp16 instead of 4-bit on the FAST route	LLM step 0.115 → 0.079 s, less memory, same F1	measured
Label checks: invoice number and date must sit on a labelled line	wrong invoices caught 42 → 64 of 95; accepted invoices fully correct 90.5 → 94.0 %; straight-through 92.5 → 86.2 %	estimated
VLM only for flagged invoices	fully correct 505 → 541 of 600 for ≈ 0.29 s extra GPU time per invoice	estimated
Route between genuinely different models	routing pays off when routes differ in cost and accuracy	design option
Redis cache and a message queue for several machines	horizontal scaling	design option

For future document-processing systems the study suggests: measure wall-clock time per stage and find the bottleneck before optimising; keep the intermediate text and validate across fields; choose numeric precision by measurement; batch dynamically; and cascade cheap and strong models instead of choosing one.

Conclusions

The thesis designed, implemented and compared two OCR → LLM architectures for extracting invoice number, date and total. It analysed the performance issues of such pipelines and reviewed OCR tools, VLMs, caching, compression and asynchronous processing (tasks 1–2). It built a sequential Architecture A and an optimised Architecture B with parallel OCR processes, an asyncio event loop, batching, a SHA-256-keyed SQLite cache, and data-driven routing between a 4-bit NF4 and an fp32 model (tasks 3–4). Both were evaluated on 600 real and synthetic invoices with three runs per configuration (task 5), and the trade-offs and improvements were discussed (task 6). The main conclusions are:

- B is 5.7 times faster and 5.7 times cheaper than A at identical accuracy (F1 0.941), using 1.9 times the GPU memory.
- The gain comes from parallel OCR and batching; the pipeline is then OCR-bound, as a simple bottleneck model predicts.
- 4-bit quantisation saves memory but no time for a 248-million-parameter model, fp16 is the better choice, and the cache helps only repeated documents.
- Fixed batching harms latency at light load, while dynamic batching gives the lowest latency at every load and keeps the throughput.
- The largest accuracy gain came from the shared extraction stage: row-wise OCR text and an arithmetic total check raised F1 from 0.85 to 0.94.
- A VLM on the same GPU is more accurate but 19 times slower; used only for uncertain invoices, it is estimated to add its accuracy at a fraction of its cost.

The optimised asynchronous architecture therefore meets the goal of the thesis. It improves throughput and cost by a factor of 5.7 and, with dynamic batching, latency as well, without any loss of accuracy.

References

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference* (Vol. 30, pp. 483–485). https://doi.org/10.1145/1465482.1465560

Anvari, Z. (2026). *From pixels to pairs: A comprehensive benchmark of LLM-driven key–value extraction in noisy document settings* (arXiv:2609.17538). arXiv.

Arte, E. (2024). *Exploring the effect of quantization on large language models for C++ programming on home hardware* [Master's thesis, Aalto University]. https://aaltodoc.aalto.fi/handle/123456789/130282

Bang, F. (2023). GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for NLP Open Source Software (NLP-OSS 2023)* (pp. 212–218). https://doi.org/10.18653/v1/2023.nlposs-1.24

Belkada, Y., & Dettmers, T. (2022). *A gentle introduction to 8-bit matrix multiplication for transformers at scale using Hugging Face Transformers, Accelerate and bitsandbytes*. Hugging Face blog. https://huggingface.co/blog/hf-bitsandbytes-integration

Beyer, L., Steiner, A., Pinto, A. S., et al. (2024). *PaliGemma: A versatile 3B VLM for transfer* (arXiv:2407.07726). arXiv.

Borchmann, Ł. (2024). *Notes on applicability of GPT-4 to document understanding* (arXiv:2405.18433). arXiv.

Chen, L., Zaharia, M., & Zou, J. (2023). *FrugalGPT: How to use large language models while reducing cost and improving performance* (arXiv:2305.05176). arXiv.

Crankshaw, D., Wang, X., Zhou, G., et al. (2017). Clipper: A low-latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)* (pp. 613–627). USENIX Association.

Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*.

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR 2023)*.

Hammarström, E. (2025). *Investigation of OCR model performance and post-OCR correction strategies: A comparative analysis* [Master's thesis, Uppsala University]. http://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-565512

He, Z., Zhang, C., Wu, Z., et al. (2025). Seeing is believing? Mitigating OCR hallucinations in multimodal large language models. In *Advances in Neural Information Processing Systems (NeurIPS 2025)*. https://arxiv.org/abs/2506.20168

Hegghammer, T. (2022). OCR with Tesseract, Amazon Textract, and Google Document AI: A benchmarking experiment. *Journal of Computational Social Science*, 5(1), 861–882. https://doi.org/10.1007/s42001-021-00149-1

Hemmer, A., Coustaty, M., Bartolo, N., et al. (2024). *Confidence-aware document OCR error detection* (arXiv:2409.04117). arXiv.

Huang, Y., Lv, T., Cui, L., Lu, Y., & Wei, F. (2022). LayoutLMv3: Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia* (pp. 4083–4091). https://doi.org/10.1145/3503161.3548112

Hugging Face. (n.d.). *Optimizing LLMs for speed and memory*. Transformers documentation. https://huggingface.co/docs/transformers/llm_tutorial_optimization

JaidedAI. (n.d.). *EasyOCR* [Software repository]. GitHub. https://github.com/JaidedAI/EasyOCR

Jin, Y., Wang, Y., Zhong, Q., et al. (2025). *Multi-stage field extraction of financial documents with OCR and compact vision-language models* (arXiv:2510.23066). arXiv.

Juusti, M. (2026). *Optimizing transformer inference throughput for batch processing* [Master's thesis, Aalto University]. https://aaltodoc.aalto.fi/handle/123456789/147320

Kim, G., Hong, T., Yim, M., et al. (2022). OCR-free document understanding transformer. In *European Conference on Computer Vision (ECCV 2022)*, LNCS 13688 (pp. 498–517). Springer. https://doi.org/10.1007/978-3-031-19815-1_29

Kwon, W., Li, Z., Zhuang, S., et al. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)* (pp. 611–626). https://doi.org/10.1145/3600006.3613165

Lamott, M., Weweler, Y.-N., Ulges, A., et al. (2024). LAPDoc: Layout-aware prompting for documents. In *Document Analysis and Recognition – ICDAR 2024*, LNCS (pp. 142–159). Springer. https://doi.org/10.1007/978-3-031-70546-5_9

Lee, K., Joshi, M., Turc, I., et al. (2023). Pix2Struct: Screenshot parsing as pretraining for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202 (pp. 18893–18912).

Li, C., Liu, W., Guo, R., et al. (2022). *PP-OCRv3: More attempts for the improvement of ultra lightweight OCR system* (arXiv:2206.03001). arXiv.

Lin, J., Tang, J., Tang, H., et al. (2024). AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems 6 (MLSys 2024)*.

Little, J. D. C. (2011). Little's law as viewed on its 50th anniversary. *Operations Research*, 59(3), 536–549. https://doi.org/10.1287/opre.1110.0940

Livathinos, N., Auer, C., Lysak, M., et al. (2025). Docling: An efficient open-source toolkit for AI-driven document conversion. In *AAAI-25 Workshop on Open-Source AI for Mainstream Use*. https://arxiv.org/abs/2501.17887

Luo, C., Shen, Y., Zhu, Z., et al. (2024). LayoutLLM: Layout instruction tuning with large language models for document understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2024)* (pp. 15630–15640). https://doi.org/10.1109/CVPR52733.2024.01480

Ma, Y., Cao, Y., Hong, Y., & Sun, A. (2023). Large language model is not a good few-shot information extractor, but a good reranker for hard samples! In *Findings of EMNLP 2023* (pp. 10572–10601). https://doi.org/10.18653/v1/2023.findings-emnlp.710

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. Section 8.3. https://nlp.stanford.edu/IR-book/

Marafioti, A., Zohar, O., Farré, M., et al. (2025). *SmolVLM: Redefining small and efficient multimodal models* (arXiv:2504.05299). arXiv.

Mindee. (2021). *docTR: Document text recognition* [Software repository]. GitHub. https://github.com/mindee/doctr

Nagayi, M., Khan, A., Frank, T., et al. (2025). *Evaluating OCR performance on food packaging labels in South Africa* (arXiv:2510.03570). arXiv.

National Institute of Standards and Technology [NIST]. (2015). *FIPS PUB 180-4: Secure hash standard (SHS)*. https://doi.org/10.6028/NIST.FIPS.180-4

NVIDIA. (n.d.). *Batchers*. Triton Inference Server user guide. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/batcher.html (accessed October 2026).

Palm, R. B., Winther, O., & Laws, F. (2017). CloudScan – A configuration-free invoice analysis system using recurrent neural networks. In *14th IAPR International Conference on Document Analysis and Recognition (ICDAR 2017)* (pp. 406–413). https://doi.org/10.1109/ICDAR.2017.74

Poznanski, J., Rangapur, A., Borchardt, J., et al. (2025). *olmOCR: Unlocking trillions of tokens in PDFs with vision language models* (arXiv:2502.18443). arXiv.

Python Software Foundation. (n.d.-a). *Event loop – asyncio*. Python 3 documentation. https://docs.python.org/3/library/asyncio-eventloop.html

Python Software Foundation. (n.d.-b). *Glossary: global interpreter lock*. Python 3 documentation. https://docs.python.org/3/glossary.html

Shen, J., Yuan, P., Ghosh, A., et al. (2026). *OCR or not? Rethinking document information extraction in the MLLMs era with real-world large-scale datasets* (arXiv:2603.02789). arXiv.

Shpigel Nacson, M., Aberdam, A., Ganz, R., et al. (2025). DocVLM: Make your VLM an efficient reader. In *Proceedings of CVPR 2025* (pp. 29005–29015). https://doi.org/10.1109/CVPR52734.2025.02701

Smith, R. (2007). An overview of the Tesseract OCR engine. In *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)* (Vol. 2, pp. 629–633). IEEE. https://doi.org/10.1109/ICDAR.2007.4376991

SQLite. (n.d.). *SQLite as an application file format*. https://www.sqlite.org/appfileformat.html

van Strien, D., Beelen, K., Ardanuy, M. C., et al. (2020). Assessing the impact of OCR quality on downstream NLP tasks. In *Proceedings of ICAART 2020* (Vol. 1, pp. 484–496). https://doi.org/10.5220/0009169004840496

Wang, P., Bai, S., Tan, S., et al. (2024). *Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution* (arXiv:2409.12191). arXiv.

Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65–76. https://doi.org/10.1145/1498765.1498785

Xu, Y., Li, M., Cui, L., et al. (2020). LayoutLM: Pre-training of text and layout for document image understanding. In *Proceedings of KDD 2020* (pp. 1192–1200). https://doi.org/10.1145/3394486.3403172

Yu, G.-I., Jeong, J. S., Kim, G.-W., et al. (2022). Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)* (pp. 521–538). USENIX Association.